

Resumen Sistemas Operativos – Segundo Teórico

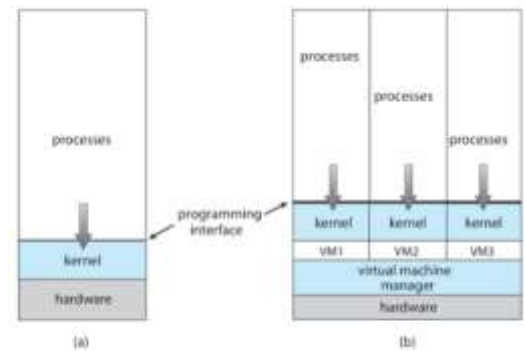
Virtualización

- Proveer recursos que tradicionalmente se asocian a hardware, pero en realidad son servicios que se comportan de igual forma.
- Son implementaciones que virtualizan pero además protegen, administran y limitan el uso del hardware.
- Las máquinas virtuales son una instancia de la virtualización, pero no es el único recurso virtualizable:
 - Storage: las máquinas virtuales utilizan discos que generalmente son archivos presentados virtualmente como dispositivos de bloque.
 - Networking: VPN, VLAN, etc.

Máquinas virtuales

- Abstraen el hardware de una maquina física en múltiples ambientes de ejecución diferentes.
- Cada ambiente genera la ilusión de correr en un hardware dedicado.
- Componentes para implementarlas:

- En la base está el host que es el que dispone del hardware que correrán las máquinas virtuales.



(a) Máquina física / (b) Máquina física, VMM y guests

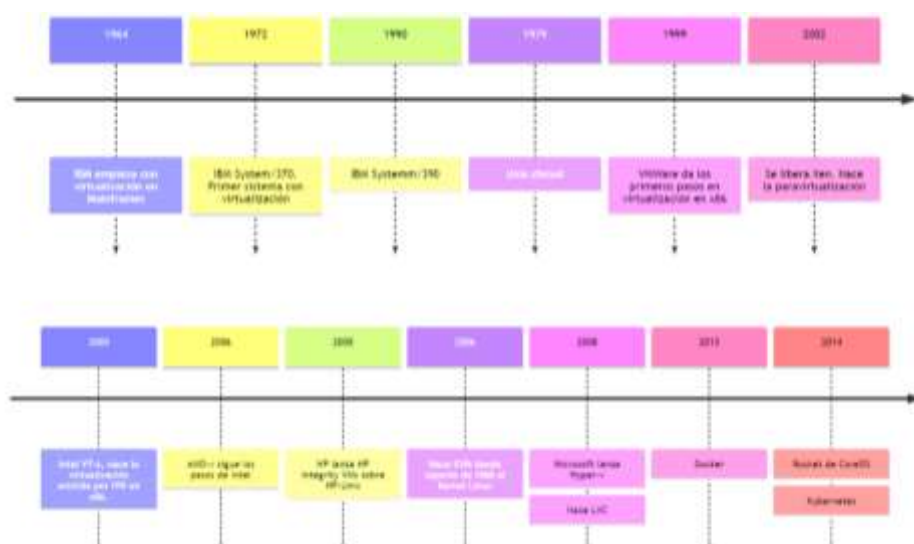
- El hypervisor o Virtual Machine Manager (VMM) creará y correrá máquinas virtuales proveyendo de una interfaz idéntica al host.
- Cada guest recibe una copia del host virtual. Los guest son las máquinas virtuales que corren un SO.

- Una máquina física puede correr múltiples SO diferentes en cada VM.

Hypervisor

- Hipervisores de tipo 0: soluciones basadas en hardware que permiten la creación y gestión de VMs a través del firmware. Tecnología legada.
- Hipervisores de tipo 1: dos clasificaciones:
 - Software dedicado que fue creado exclusivamente para ofrecer ambientes de virtualización o VMM
 - SO de propósito general como Windows o Linux que ofrecen capacidades de virtualización, es decir son VMM, y además un clásico SO.
- Hipervisores de tipo 2: aplicaciones que corren sobre un SO estándar y confían en el SO preexistente para manejar los recursos virtuales. Introducen latencia.
- Paravirtualización: el SO guest debe modificarse para trabajar en cooperación con el VMM.
 - Mejor performance.
 - Alta cohesión entre guest y el VMM.
- Ambientes virtualizados de Programación: el VMM no virtualiza hardware real, sino que por el contrario crea un sistema virtual optimizado. La principal diferencia con los anteriores es que se utiliza para correr aplicaciones y no SOs.
- Emuladores: permite correr aplicaciones escritas para un hardware determinado en un ambiente completamente diferente.
- Contenedores: características similares a la virtualización independizando las aplicaciones del SO subyacente. Facilita el empaquetar, distribuir y ejecutar aplicaciones que autocontienen sus dependencias. La unidad de ejecución es una aplicación.

Historia



Ventajas de la virtualización

- Usar el mismo hardware para correr distintos SO de manera concurrente sin dual boot.
- El host está protegido de cada VM y estas están protegidas entre sí.
- Desarrollo o estudio de sistemas operativos: permite realizar pruebas sin afectar al host.
- Durante el desarrollo de aplicaciones:
 - Permite verificar la portabilidad o funcionamiento de un producto en nuevas versiones de SO.
 - Armar ambientes complejos utilizando VMs que corren servicios que son parte de arquitecturas grandes.
- Al trabajar con infraestructura como código: el código es probado en diferentes plataformas usando virtualización.
- Consolidación de sistemas físicos en virtuales: la conversión de sistemas físicos en VMs ofrece una reducción de espacio y optimización de recursos.
- Templates: modelo de VM creado con personalizaciones para poder crear nuevas VMs a partir de él con menor esfuerzo de configuración.

Desventajas

- La forma en que las VMs comparten recursos entre sí:
 - Dos formas:
 - Usando filesystems compartidos y a través de ellos compartiendo archivos.
 - Usando una red de máquinas virtuales donde cada una de ellas se comunica.
- VM sprawl: el número de VMs en una red crece al punto de volverse inmanejable.

Ciclo de vida de las VMs

- Stopped: una VM que no se está ejecutando.
- Running: una VM que se está ejecutando.

- Suspended: similar a poner una maquina en hibernación. Se almacena disco y estado (memoria, registros, etc.) de VM.

Snapshots

- Tomar una instantánea del storage de una VM en un momento mientras continúa su ejecución.
- Al poder freezar el filesystem de una virtual usando snapshots:
 - Se puede migrar de un host físico a otro.
 - Clonar una VM a partir de otra.
 - Tomar un resguardo antes de realizar un cambio y así poder disponer de point in time recovery.
 - Política de backup (no recomendable)

Live migration

- Mover VMs en estado running desde un servidor físico a otro sin interrumpir su operación o conexiones de red que mantenga activas.
- Suele usarse para liberar de carga a un host que se encuentre sobrecargado sin interrumpir el servicio del guest que se migre.
- Permite realizar mantenimiento sobre HW físico moviendo sus guests para mantenimiento.

Estándares portables

- En 2007 se estandariza el Open Virtualization Format u OVF.
- Consta de un conjunto de archivos ubicados en un directorio:
 - Un descriptor con extensión .ovf que es un XML con descripciones de la VM.
 - Imágenes de discos, certificados y otros archivos auxiliares.
- El directorio completo se distribuye como un paquete Open Virtual Appliance u OVA.
 - Es un TAR con el contenido del directorio OVF.

Otros usos de la virtualización

- Cloud Computing
 - Las plataformas Cloud utiliza la virtualización para ofrecer sus servicios basándose en APIs que permiten a los usuarios gestionar la infraestructura por demanda.
- Virtualizar desktops en puestos de trabajo para ahorrar gastos.

Técnicas de virtualización

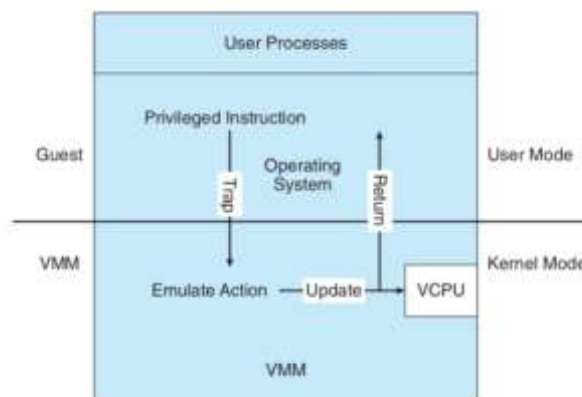
- Problemas
 - No todos los procesadores discriminan entre instrucciones privilegiadas y no privilegiadas.
- Un artículo superduper importante dice que una VMM debe ofrecer:
 - Fidelidad: los programas que corren en una VM lo hacen igual que de forma nativa.
 - Performance: una buena cantidad de instrucciones del guest se ejecutan por el HW sin intervención del VMM.
 - Seguridad: el VMM maneja el acceso al HW.
 - Además describe características que debe cumplir el conjunto de instrucciones de la arquitectura (ISA) para implementar un VMM.
 - Clasificación de instrucciones ISA:
 - Privilegiadas: irrumpen cuando la CPU está en modo usuario. No si está en modo kernel.
 - Sensibles: intentan modificar la configuración de recursos del sistema directa o indirectamente: deshabilitar interrupciones, cambiar el timer, etc.
- El teorema del artículo enuncia que cualquier computador de 3er generación podrá construir un VMM si el conjunto de instrucciones sensibles es un subconjunto de las instrucciones privilegiadas.
 - Entonces para construir un VMM las instrucciones sensibles y privilegiadas tienen que generar una interrupción que sea capturada por el VMM.
 - Las instrucciones no privilegiadas pueden ejecutar nativamente.

Virtual CPU o vCPU

- No ejecutan código sino que para cada guest el VMM mantiene una vCPU representando el estado actual de cada uno.
- Cuando el VMM debe cambiar el contexto de una VM a la CPU, la información de la vCPU es usada para cargar el contenido adecuado.

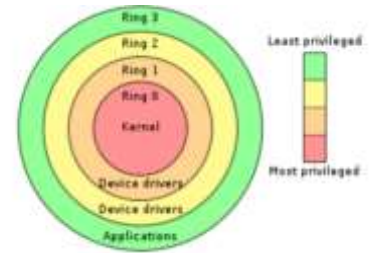
Trap and emulate

- Conocida como virtualización clásica.
- Las instrucciones privilegiadas corren en modo kernel y las no privilegiadas en modo usuario. Las máquinas virtuales guest se ejecutan en modo usuario.
- La VM tiene un virtual user mode y un virtual kernel mode:
 - Una VM pasa de virtual user mode a virtual kernel mode.
 - Cuando el kernel en el guest intenta ejecutar una instrucción privilegiada o sensible, genera un trap (porque el sistema real está en modo usuario):
 - Este es capturado por el VMM en el host. El VMM gana el control y ejecuta la acción iniciada por el guest en su nombre y luego retorna el control a la VM guest.
- Las instrucciones privilegiadas sufren una mínima latencia. Las instrucciones no privilegiadas no ya que pueden correr nativamente en el HW.



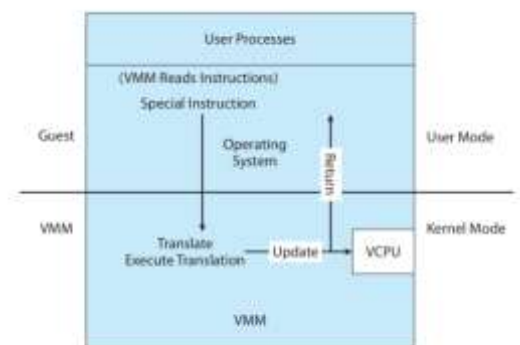
La arquitectura x86

- Niveles de privilegio en x86
 - En ring 0 ejecuta el kernel del SO y drivers de dispositivos
 - En ring 3 las aplicaciones.
 - Los rings 1 y 2 no son típicamente usados por el SO.
- Problema:
 - No se cumple el teorema ya algunas instrucciones sensibles no son privilegiadas. Aproximadamente 17 instrucciones sensibles no son privilegiadas.
 - Entonces no se genera un trap cuando se es una instrucción sensible ya que no es considerada privilegiada.
 - Si no hay trap, el VMM no se enterará cuando una VM intenta ejecutar la instrucción.
 - Por esto, trap and emulate no funciona en x86.



Binary translation

- Difícil de implementar.
- Si la vCPU de un guest se encuentra en modo usuario puede correr instrucciones de forma nativa en la CPU física.
- Cuando la vCPU del guest se encuentre en modo kernel:
 - El VMM examina cada instrucción que ejecuta el guest en virtual kernel mode, adelantándose a las siguientes instrucciones que el guest ejecutará (basándose en el contador del programa de la VCPU).
 - Las instrucciones especiales son traducidas a un nuevo conjunto de instrucciones que realizan una tarea similar.
- Performance mala.
- Instrucciones que no se ejecutan de forma nativa agregan mucha latencia porque una única instrucción es reemplazada por varias.



- Mejora: uso de caché con traducciones recientes.

Manejo de la memoria

- Se utiliza NPT (Nested Page Tables).
- Cada guest SO mantiene una o más tablas de paginación para traducir de memoria virtual a memoria física.
- El VMM mantiene NPTs que representan el estado de la tabla de paginación de los guests.
- El VMM detecta cuando el guest intenta cambiar su tabla de paginación y realiza un cambio equivalente en el NPT.
- Cuando un guest se encuentra ejecutando en la CPU, el VMM pondrá el puntero al NPT correspondiente en el registro apropiado de la CPU, de tal forma que la tabla sea la tabla de paginación activa.
- Cuando un guest necesita modificar la tabla de paginación la operación debe ser interceptada por el VMM y realizar las modificaciones necesarias en el NPT y las tablas de paginación del sistema.
- Esto generará un incremento en las fallas del TLB y más problemas que se deben manejar.

Virtualización asistida por HW en x86

- Surgen procesadores Intel que ofrecen soporte nativo de virtualización proveyendo las instrucciones VT-x y procesadores AMD a través de la tecnología AMD-V.
 - Ambos definen dos nuevos modos de operación: host y guest
 - El uso de binary translation no se necesita más.

Cómo funciona la virtualización en x86

- El VMM ahora puede habilitar el modo host, definir las características de cada máquinas virtual, y luego cambiar el sistema al modo guest, pasándole el control al SO guest:
 - El SO virtualizado creará estar corriendo en hardware nativo.

- Si el SO intenta acceder a un recurso virtualizado, el control es pasado al VMM para manejar esta interacción.
- Otra mejora es la implementación por HW del concepto de NPT.
 - Intel la llama EPT (Extended Pages Tables).
 - AMD RVI (Rapid Virtualization Indexing).

Virtualización de dispositivos

- El acceso a dispositivos de I/O es otro área que fue mejorada por la asistencia del HW.
- IOMMU o I/O MMU Virtualization
 - Permite a las máquinas virtuales guests utilizar directamente dispositivos físicos a través de DMA y remapeo de interrupciones.
 - También suele llamarse esta tecnología como PCI Passthrough.
 - AMD llama a esta tecnología AMD-Vi
 - Intel la llama VT-d.

Contenedores

Contenedores de aplicaciones

- Empaquetan aplicaciones en una unidad estándar de intercambio.
- Única pieza de software en un filesystem completo que contiene todo lo necesario para ejecutar una aplicación.
- Garantiza que el software siempre correrá de igual forma sin importar su ambiente. Tiene una parte rígida y otra mutable, que es parametrizada.
- En los contenedores se debe tener el mismo sistema operativo.

¿Por qué usar contenedores?

- Minimiza diferencias entre ambiente de desarrollo y producción.
- Simplifica la instalación de aplicaciones en diferentes plataformas.
- Despliegue de aplicaciones complejas.
- Empaquetado de aplicaciones legadas.

Características de los contenedores

- Flexibles: incluso las aplicaciones más complejas pueden conteneirizarse.
- Livianos: máximo aprovechamiento del kernel por ser compartido por todos los contenedores y el sistema.
- Portables: construcción local, desplegables en la nube y aptos para correr en cualquier plataforma.
- Débilmente acoplados: los contenedores son autosuficientes y encapsulados.
- Escalables: se pueden incrementar o disminuir la cantidad de contenedores para un servicio.
- Seguros: hay aislamiento.

Precedentes

- chroot: para aislar un filesystem.
 - chroot aísla filesystem, no procesos.
- namespaces: para dar garantías de las cuales carece chroot:
 - Aislar procesos
 - La red
 - Puntos de montaje
 - La forma de lanzar un proceso asilando namespaces es mediante el comando unshare.
- cgroups: para acotar los recursos consumidos por los contenedores contenedor. Hay dos versiones de cgroups: v1 y v2. Las nuevas distribuciones Linux proveen cgroups v2.
 - Los procesos deben hacer caso a los límites impuestos, para así no caer en, por ejemplo, OOMKills.

LXC - Linux Containers

- LXC es parte de LXD, un daemon para la gestión de ambientes virtuales usando virtualización con qemu y contenedores.
- Basado en el concepto de imágenes remotas que se descargan e instancian en alguna de las tecnologías indicadas.

- Se utilizan perfiles para configurar ciertas características de las instancias.
- Su uso se asocia más a la gestión de máquinas como si fueran virtuales.
- La filosofía detrás de LXD es correr ambientes virtuales.

Docker

- Se basa en el uso de:
 - cgroups para restringir recursos.
 - kernel namespaces para aislar información.
 - filesystem de unión.
- Se obtienen contenedores independientes en una instancia Linux que evita el overhead de manipular VMs.
- Antes de la versión 0.9, Docker usaba LXC luego incorporaron libcontainer.
- Un contenedor es un proceso corriendo con algunas características de encapsulamiento aplicadas para mantenerlo aislado de otros contenedores e incluso del sistema base.
- Cada contenedor mantiene un filesystem propio provisto por una imagen.
- Imagen: filesystem y parámetros para utilizarla. No cambia nunca y no tiene estados.
- Contenedor: instancia de una imagen. Tiene una capa de RW volátil.

docker cli

- `docker run`
 - corre un contenedor a partir de una imagen.
 - `-i` mantiene abierto STDIN. Es el modo interactivo.
 - `-t` creará una pseudo-tty. Combinado con la opción `-i` el uso del contenedor es similar al utilizar una consola ssh, xterm o shell.
 - `-e` setea una variable de ambiente.
 - `-rm` elimina el contenedor cuando termina.
 - `--name` nombrar al contenedor
 - Exit code exitoso es 0.
- `docker image ls`
 - listar las imágenes disponibles en nuestro sistema.

- `docker ps -all`
 - listar contenedores
- `docker ps -qf status=exited | xargs docker rm`
 - eliminamos los contenedores finalizados
 - `-f` para filtrar contenedores por diversos criterios.

Imágenes

- Imágenes se descargan de Docker Hub.
- Se pueden descargar con `run` o `pull`.
- `inspect`: para inspeccionar toda la información acerca de un contenedor.
- `exec`: permite acceder a un contenedor corriendo.
 - No es una buena práctica abrir un puerto para acceder por ssh.
- Se pueden construir de forma imperativa una imagen usando `commit` a partir de un contenedor (corriendo o terminado).

Políticas de reinicio

- Permiten especificar cómo un contenedor debería o no ser reiniciado cuando termina.
- Para definir la política, `docker run` admite la opción `--restart`.
- Consideraciones para tener presentes respecto de las políticas:
 - El contenedor debe mantenerse corriendo por al menos 10 segundos y Docker pueda monitorizarlo. Previene un loop infinito.
 - Si un contenedor se para con `stop` su política de reinicio quedará sin efecto hasta que se reinicie.
 - Las políticas aplican sólo a contenedores.
- Políticas:
 - `no`: no iniciar el contenedor cuando termina. Por defecto.
 - `on-failure:[max]`: reiniciar solo si el contenedor termina con exit status diferente a cero. Limitar opcionalmente los reintentos de reinicio.
 - `always`: siempre reiniciar el contenedor. Además el contenedor se iniciará cuando inicia el daemon Docker. Incluso aquellos que se hayan parado con `docker stop`.

- unless-stopped: idem anterior, salvo que en un reinicio del servicio Docker no iniciará aquellos contenedores que se hayan parado con docker stop.

Dockerfile

- Construcción de imágenes de forma declarativa.
- Utiliza un archivo de texto plano para definir el contenido de una imagen.
- Permite escribir instrucciones a ejecutar.
- Automatiza el proceso de la creación de imágenes.
- Permite repetir y modificar fácilmente una imagen.
- Generar de forma simple imágenes derivadas.

History

- Permite ver como se construyeron las capas. Según como se esté construyendo con esto se pueden ver cosas viejas y vulnerar la seguridad.

CMD y ENTRYPOINT

- Permiten especificar qué comando se ejecutará de forma predeterminada al instanciar una imagen.
- Dependiendo de cómo se empleen pueden tener diferente significado
- Utilizar CMD en el Dockerfile cuando se desea flexibilizar al usuario la modificación del CMD que provee nuestra imagen.
- ENTRYPOINT se utiliza cuando los contenedores se deben comportar de forma exclusiva. Para que los usuarios de la imagen no modifiquen el ejecutable definido.
- Las instrucciones CMD y ENTRYPOINT soportan dos formatos llamados:
 - SHELL: especificando los comandos como string
 - CMD executable param1 param2
 - EXEC: especificando los comandos como arreglos de strings
 - CMD ["executable","param1","param2"]
 - El formato recomendado siempre es EXEC porque SHELL puede llevar a errores.

- Las instrucciones ENTRYPOINT y CMD pueden combinarse.

PID 1

- Importante cual es el proceso que queda corriendo con el PID 1.
- Este suele ser el proceso init que tiene otras responsabilidades, como por ejemplo adoptar procesos huérfanos y manejar zombies.
- Un proceso que recibe la señal SIGTERM y no implementa un manejador termina de forma inmediata.
 - Si el PID 1 recibe esta señal y no implementa un manejador, no hará nada.

Registry

- Servicio para almacenar y distribuir imágenes de Docker.
- Permite el guardado de imágenes docker referenciadas por un nombre y un tag.
- Los usuarios interactúan con la registry mediante pull y push.
- Docker Hub: gratis para imágenes públicas. Limita a cantidad de peticiones a la API.
- RedHat Quay: sin cargo para imágenes públicas.
- GitHub Packages: sin costo por el momento.
- Gitlab Container Registry: sin costo por el momento, pero con cuota de espacio.

Volúmenes

- No utilizan UFS.
- Más performantes que utilizar la capa RW del contenedor.
- Pueden compartirse y reusarse entre contenedores.
- Los cambios se hacen directamente en el volumen.
- La información no se incluye en la imagen.
- Persisten aun cuando se eliminen todos los contenedores que los usan.
 - Pueden quedar volúmenes sin referenciar.
- Tipos:
 - Bind mounts: son directorios del propio filesystem en el SO host.

- Pueden utilizar cualquier directorio del sistema host. Incluso podrían importar directorios importantes.
- Pueden especificar bind propagation
- Se oculta la información que exista en el punto de montaje.
 - Correspondencia con el comando mount.
- Volúmenes manejados por Docker.
 - Pueden ser anónimos o nombrados.
 - Se almacenan /var/lib/docker/volumes
 - Son la mejor forma de persistir datos en Docker.
 - Se manejan mediante docker cli.
 - Funcionan en Linux y Windows.
 - Pueden compartirse de forma más segura entre diferentes contenedores.
 - Se pueden almacenar volúmenes en hosts remotos o servicios de la nube, encriptar su contenido, etc.
 - Un nuevo volumen puede inicializarse con datos por un contenedor.
 - Al montar un volumen la información que exista en el punto de montaje se copia al volumen.
- Si se utiliza Linux también pueden utilizarse montajes de tipo tmpfs.
 - Se almacenan en memoria del sistema host y nunca escriben en archivos.
 - Usado internamente por los servicios swarm para montar secrets.
 - Si un contenedor utilizará información que no deba persistirse se recomienda utilizar tmpfs

Volúmenes

- Se usaba -v ó --volume para correr contenedores standalone, es decir, con docker run y en modo swarm, los servicios usaban --mount.
- Hoy en día --mount puede ser usada por contenedores standalone:
 - Son el mecanismo preferido y recomendado por su expresividad y simplicidad
 - -v combina todas las opciones en un campo mientras que --mount los separa.

Gestión de recursos

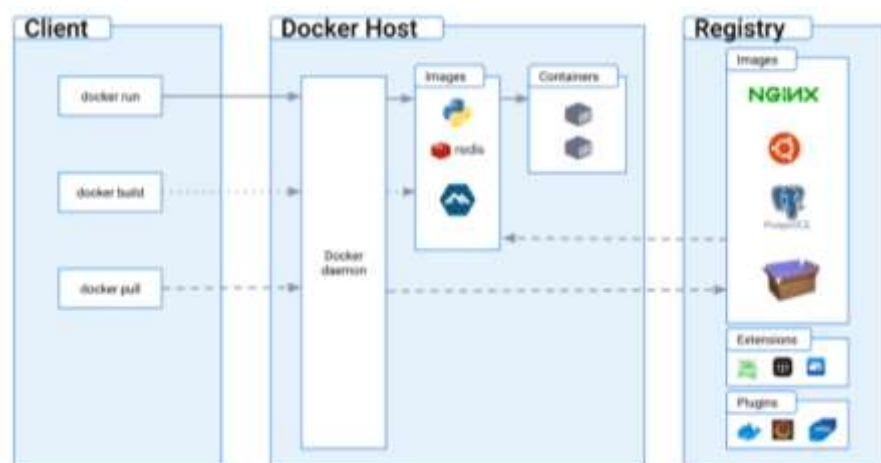
- Posible controlar la cantidad de recursos que puede consumir un contenedor mediante cgroups.
- Para limitar la memoria hay que deshabilitar la swap porque si no los contenedores la van a usar cuando no tengan suficiente memoria.
- Limitando los PIDs se puede limitar cuantos procesos inicia un contenedor.

Docker Daemon

- dockerd es el servidor que presta Docker.
- El socket donde recibe comandos utiliza un socket unix en /var/run/docker.sock. Se puede cambiar.
- Se puede configurar con opciones al daemon o mediante el archivo de configuración /etc/docker/daemon.json.

Contenedores sin Docker

- Docker es monolítico, una de las razones para no ser usado:
 - Construye las imágenes
 - Push/Pull de imágenes a/de registries
 - Corre las imágenes en contenedores
 - El servicio de docker hace todo, si falla el servicio, todos sus contenedores serán inalcanzables.



- Existen alternativas a Docker daemonless como:
 - podman
 - Su uso es tal cuál docker. Es más, en algunos textos, puede encontrarse la sugerencia de usar alias `docker=podman`.
 - Corre en modo usuario a diferencia de Docker que se necesitan privilegios de root.
 - Mas seguro, ya que no permite abrir puerto privilegiado.
 - Se pueden construir imágenes con `podman build`.
 - Buildah
 - Se especializa en la construcción de imágenes OCI.
 - Es una alternativa para cada uno de los comandos que se encuentran en un Containerfile.
 - Permite la construcción de imágenes sin privilegios de root.
 - Puede usarse desde cualquier lenguaje de scripting para construir imágenes sin Containerfile.
 - Podman usa librerías de buildah para construir imágenes a partir de Containerfile.

Orquestación de contenedores

- La idea de los clusters de contenedores es disponer de nodos físicos con algún runtime que les permita correr contenedores.
- Características:
 - Diseño descentralizado.
 - Tolerantes a fallas.
 - Alta disponibilidad.
 - Balanceo de carga.
 - Roles definidos.
 - Master..
 - Workers
 - Servicios o pods.
 - Scheduling.
 - Distribuir la ejecución de contenedores entre un pool de workers.
 - Recuperación ante fallas.
 - Rebalanceo de contenedores cuando fallan workers.

- Escalamiento de contenedores.
- Health checks.
 - Determinar el estado de servicios.
 - Autohealing.
- Service Discovery.
 - Crear nuevos servicios no tiene sentido si no es simple encontrarlos.
- Balanceo de requerimientos entre servicios internos o de ingreso al cluster.
- Attach de varios tipos de storage a contenedores dentro del cluster.
- Hoy día el cluster de contenedores más popular es Kubernetes.

Kubernetes

- Plataforma portable y extensible de código abierto para administrar cargas de trabajo y servicios.
- Facilita la automatización y la configuración declarativa.
- Tiene un ecosistema grande y en rápido crecimiento.
- Google transfiere la propiedad de k8s a la fundación CNC.
- CNCF – Cloud Native Computing Foundation
 - Las tecnologías CN empoderan a las organizaciones para construir y correr aplicaciones escalables en ambientes heterogéneos.
 - Permite crear sistemas con bajo acoplamiento que son resilientes, manejables y observables, facilitando cambios frecuentes y predecibles.
- La CNCF promueve este paradigma apoyando un ecosistema de proyectos de código abierto y neutrales respecto a proveedores.
- Criterio de graduación CNCF
 - El criterio de graduación de la CNCF define las etapas para la adopción de productos tecnológicos:
 - Sandbox Stage: necesita sponsors, respetar el código de conducta de CNCF, adherirse a la política de propiedad intelectual y mostrar su estado sandbox.

- Incubating Stage: igual que sandbox, pero con al menos tres usuarios independientes en producción, contribuciones garantizadas y un esquema de versionado claro.
- Graduated Stage: igual que incubating, pero con desarrolladores de al menos dos organizaciones diferentes, un listado público de implementaciones de referencia, una Core Infrastructure Initiative, gobernanza explícita del proyecto, proceso de aceptación de contribuciones y la mayoría de los votos del comité.

Arquitectura

- Cluster: conjunto de nodos que se comportan como un único sistema.
- Nodo del cluster: cualquier máquina, física o virtual, que se encuentre corriendo los servicios de k8s y se haya registrado en el cluster.
- Cluster persiste su estado en una base de datos distribuida
- Kubernetes: es un sistema de código abierto para automatizar el despliegue, escalamiento y administración de aplicaciones containerizadas y servicios.
 - No es una plataforma como servicio.
 - Provee una capa de abstracción al hardware para interactuar con aplicaciones en contenedores.
 - No construye ni despliega el código (CI/CD).
 - No provee servicios a nivel de capa de aplicación.
 - No provee servicios dedicados al monitoreo.
 - No provee un lenguaje propio para la configuración.
 - Es mucho más que un orquestador de contenedores:
 - Orquestación es la ejecución de un flujo definido:
 - Primero ejecutar A, luego B luego C.
 - K8S provee diferentes procesos de control que se componen y actúan de forma independiente para llevar el estado actual al deseado
- Arquitectura base:
 - Roles: un mismo nodo puede cumplir más de un rol al mismo tiempo.

- Master.
- Worker.
- etcd.
- Todos los componentes se comunican usando APIs siguiendo el estandar OpenAPI.
- Un pod es la unidad más pequeña de despliegue y administración. Encapsula uno o más contenedores que comparten recursos de almacenamiento y red, y especificaciones sobre cómo ejecutarse. Permiten que los contenedores se comuniquen entre sí y se gestionen juntos. Facilitan la escalabilidad.

Servicios en nodos master

- Cluster Control Plane: toma decisiones globales
 - kube-apiserver.
 - Exponen la API de k8s.
 - Es el frontend del cluster.
 - Comandos interactúan con él para operaciones.
 - kube-scheduler.
 - Observa la creación de nuevos pods y seleccionando un nodo para su ejecución.
 - kube-controller-manager: es un binario con varios controladores.
 - Node controller: notifica cuando un nodo deja de responder.
 - Replica controller: asegura que un número específico de Pods idénticos estén ejecutándose en todo momento.
 - Endpoints controller: une servicios con pods permitiendo que el tráfico de red dirigido a un servicio sea redirigido a los pods que implementan dicho servicio.
 - Service account & Token controllers: crea cuentas por defecto y tokens de acceso a la API para los nuevos namespaces.
 - cloud-controller-manager: gestiona la interacción con los servicios del proveedor de cloud:
 - Node controller: detecta si un nodo ha sido eliminado cuando deja de responder.
 - Route controller: configura rutas en la red.
 - Service controller: configura balanceadores de carga.

- Volume controller: crea, asocia y monta volúmenes.

Servicios en todos los nodos

- Los siguientes servicios corren tanto en nodos master como en workers:
 - Kubelet:
 - Asegura que los contenedores estén corriendo en un pod.
 - No gestiona contenedores que no son creados por k8s.
 - kube-proxy
 - Mantiene reglas de red en los nodos, permitiendo así la comunicación con los pods correctos desde dentro y fuera del cluster.

Servicios en nodos etcd

- Corren el servicio de etcd, una base de datos clave-valor distribuida.
 - etcd almacena toda la información y el estado del cluster.

Container runtimes

- Para correr contenedores en pods, Kubernetes utiliza un container runtime específico. Algunos de ellos:
 - ~~Docker~~
 - Ya no es usado.
 - cri-o
 - containerd
- En 2015 docker junto a otras empresas lanzan OCI (Open Container Initiative) que define dos especificaciones.
 - Runtime Spec: con runC como referencia de implementación que es un runtime de bajo nivel que no provee una API.
 - Image Spec: define un estándar para la construcción y transporte de imágenes.
- Containerd actúa como un API facade del container runtime (runc en este caso)
- CRI permite a kubelet usar una gran variedad de container runtimes.

- CRI-O es un container runtime liviano de alto nivel creado específicamente para k8s y soporta runtimes de bajo nivel como runC.

Redes

- Los nodos de un cluster pueden estar distribuidos y en redes IP diferentes. Los Pods deben conectarse entre sí, incluso en diferentes nodos. Esto requiere asignarles una IP de una red, dependiendo de la implementación utilizada.
- Se utilizan tecnologías como VXLAN, IPIP, GRE y BGP u OSPF para el encapsulamiento y distribución de rutas.
- Network plugins:
 - Flannel.
 - *No quise poner más.*
 - Estos plugins no solo proporcionan conectividad entre pods en diferentes nodos, sino que también implementan Network Policies para limitar el tráfico entrante o saliente de los pods.

Alta disponibilidad

- Propiedad deseable.
- Mínimos requisitos:
 - 3 nodos etcd.
 - 3 nodo master (pueden ser los mismos de arriba).
 - 2 nodos con rol worker.

Capacidad de kubernetes

- No más de 5.000 nodos.
- No más de 150.000 pods.
- No más de 300.000 de contenedores.
- No más de 110 pods por nodo.

Instalaciones

- Tipos de instalaciones:
 - On-premise o Baremetal: se gestionan todos los nodos y se usa el cluster para desplegar aplicaciones.
 - Cloud providers:
 - Como On-premise pero usando servicios de la nube y aprovechando el escalamiento de workers.
 - Managed kubernetes: se eligen tipo de máquinas para los workers, pero los masters y etcd son gestionados por el proveedor de cloud.
- Instalación para dev / desktop
 - kind: kubernetes in docker.
 - etc.

Objetos de kubernetes

- Entidades persistentes dentro del sistema k8s.
- Representan el estado del cluster.
- Describen:
 - Las aplicaciones conteneirizadas corriendo.
 - Recursos disponibles.
 - Políticas de las aplicaciones: políticas de reinicio, actualizaciones, y tolerancia a fallas.
- En formato JSON o YAML.
- Objeto consiste de las siguientes secciones:
 - apiVersion.
 - La API de k8s se actualiza continuamente por lo que se versiona permitiendo que un cluster k8s mantenga objetos con formatos diferentes. Con el versionado se pueden agregar, eliminar o reestructurar objetos.
 - Se simplifican las actualizaciones del cluster.
 - Toda la interacción se realizará usando https. El cliente propio de kubernetes es kubectl.
 - kind.
 - metadata.
 - name.
 - labels.

- son utilizados para buscar objetos que machean con los labels para realizar autodiscovery.
 - annotations.
 - son usadas para almacenar metadatos, pero no es posible buscar por este campo.
 - uid.
 - spec.
- Del kind y apiVersion dependerá la morfología de un objeto. La misma estará en el campo spec o data, dependiendo de cada objeto.
- Namespaces:
 - Los objetos pueden ser globales o crearse en un namespace.
 - Es una forma de separar objetos para agruparlos y aplicar políticas de seguridad a cada uno de forma independiente.
 - El namespace es un objeto global.
- Objetos más relevantes:
 - Pod: unidad mínima de ejecución en Kubernetes, representando una carga de trabajo. Puede tener uno o más contenedores.
 - Deployment: controla el despliegue y actualización de pods, evitando downtime. Usado para servicios stateless y permite escalar fácilmente.
 - Statefulset: similar deployments pero para workloads statefull.
 - Daemonset: asegura que un pod se ejecute en cada nodo del cluster.
 - Job: ejecuta tareas que finalizan, pudiendo correr en paralelo.
 - Cronjob: ejecuta jobs periódicamente y de forma recurrente.
- Objetos que permiten configurar contenedores:
 - Configmap: repositorio de valores en formato clave valor.
 - Secrets: similar al configmap, pero con la diferencia que los datos se codifican en base64.
- Conectividad entre contenedores:
 - Service: un servicio buscará pods usando labels y de esta forma balancear el tráfico que ingrese al pod.
 - Ingress: permiten ingresar tráfico al cluster usando un balanceador HTTP.
- Persistencia:
 - PersistentVolumes (PV): Volúmenes persistentes globales montados por contenedores en un Pod. Soportan varios tipos de volúmenes.

- PersistentVolumeClaims (PVC): Solicitudes de PVs que pertenecen a un namespace específico.

Ejecutables

Archivos

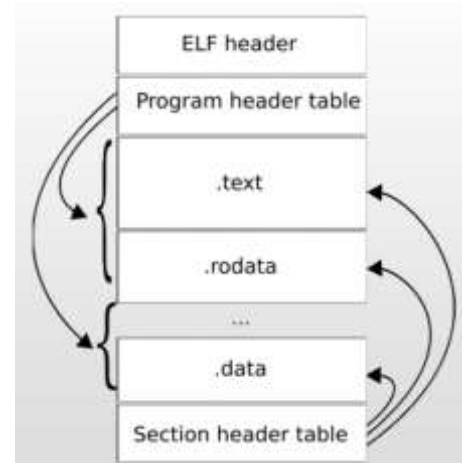
- Es un tipo abstracto de datos que manipula su información a través de una interfaz conocida y bien definida.
- El tipo de dato abstracto define la forma en la que se va a agrupar la información dentro del archivo.
- La interfaz define el conjunto de operaciones que se puede realizar sobre los archivos.
- Existen distintos tipos de archivo cada uno con un formato para representar los datos.

ELF

- Executable and Linking Format es un formato de archivo para ejecutables, código objeto, bibliotecas compartidas y volcado de memoria.
- Se transformó en un standard en formato de archivo de los sistemas Unix Like.
- ELF es parte de la "System V application binary interface" (ABI), que define una interfaz del sistema operativo para operar con programas compilados ejecutables, y sus funciones.
- Si sigue ABI puedo ejecutar archivo en sistema Unix Like (si las condiciones están dadas).
- Capacidades:
 - Soporta dynamic linking.
 - Soporta dynamic loading.
 - Facilita la creación de librerías compartidas.
 - La representación del archivo es independiente de la plataforma.
- Object File

- Son representaciones binarias de programas que se ejecutan directamente en el procesador. No hace pasarlo por un intérprete o preprocesador.
- Los crea el ensamblador y el link editor.
- Assembler puro.
- Participan en el linking de los programas y la ejecución.
- Se excluyen los programas que requieren un intérprete.
- Tres tipos principales de Object Files:
 - Executables: programa que puede ser ejecutado.
 - Es un formato para archivos aptos para ejecución
 - `gcc -o program program.c`
 - Por defecto en gcc cuando no se indica ningún parámetro adicional al `-o` (output), lo que se genera es un archivo ejecutable.
 - Relocatable: código y datos que puede ser utilizado para linkarse con otros object files para formar un ejecutable o un shared object file.
 - Es la “base” para crear ejecutables y Librerías.
 - `gcc -c program.c`
 - Son como una especie de código intermedio que se va creando para poder crear nuevos archivos ejecutables o shared objects.
 - Generalmente archivos `.o` o `.ko`.
 - El parámetro `-c` indica al gcc que no ejecute al linkeditor.
 - Shared Object: código y datos para ser utilizados en dos contextos:
 - En el momento de compilación por el linkeditor.
 - En el momento de ejecución por el linkeditor.
 - Son las librerías compartidas. Muy sutil la diferencia entre relocatable y shared object.

- Generalmente archivos .so
 - /usr/lib
- Secciones del ELF:
 - Cada archivo ELF está formado por un Header y sus datos:
 - Header
 - Siempre al comienzo.
 - Tiene un "road map" que describe la organización del archivo
 - Único y tiene lugar fijo.
 - Define si se utilizan direcciones de 32 o 64 bits.
 - Dentro de sus campos se encuentran:
 - Magic Number.
 - Endianness (big o Little).
 - Plataforma para la que fue compilado.
 - Arquitectura para la que fue compilado.
 - Etc.
 - Es un array de estructuras.
 - Tiene, por ejemplo, identificación del archivo, tipo, arquitectura, punto de entrada, donde comienzan las Program y Header tables y cantidad de entradas de estas, etc.
 - Program Header Table
 - Tabla que indica como se debe crear la imagen del proceso.
 - Se lee en tiempo de ejecución.
 - En el Header se indica dónde está el comienzo de la tabla.
 - Opcional.
 - Los ejecutables la deben tener siempre.
 - Los relocatables no la necesitan.
 - Es un array de estructuras
 - Tiene, por ejemplo, tipo de segmento, desplazamiento de comienzo dentro del archivo,



dirección virtual donde se encontrará, tamaño, flags (se puede leer, escribir, ejecutar, etc.).

- Section Header Table
 - Contiene información que describe las secciones del archivo y permite localizarlas.
 - Cada sección tiene una entrada en la tabla, con información de esta.
 - Los archivos que se usan durante el linking (relocatables principalmente) la deben tener los object pueden no tenerla.
 - Es un array de estructuras.
 - Tiene, por ejemplo, el nombre de la seccion, tipo, comienzo en el archivo, tamaño, flags (se puede leer, escribir, ejecutar, etc.).
- Información: ELF sections y ELF segments:
 - Son divisiones Lógicas dentro de archivo que agrupan la información.
 - Las secciones son utilizadas durante el linking.
 - Como construir el programa.
 - Los segmentos son utilizados durante la ejecución.
 - Como ejecutarlo.

Secciones

- Colección de información de tipo similar.
- Cada sección representa una porción del archivo.
- Contienen un nombre y un tipo.
- Ejemplos de secciones especiales
 - .text: Instrucciones ejecutables.
 - .data: variables inicializadas por el usuario.
 - .bss: Datos no inicializados.
 - .comment: Información sobre control de versiones
 - .strtab: Strings que representan los nombres asociados en la Symbol Table
- Corresponde con la Section Header.

Segmentos

- Se utilizan y resuelven en tiempo de ejecución.
- El Kernel los utiliza para ejecutar un programa.
- Un Segment puede corresponderse con una o varias sections.
 - Agrupa secciones de acuerdo a su naturaleza.
- El Kernel mantiene estructuras VMA (Virtual Memory Area) para cada proceso:
 - Se corresponden con un segmento.
 - Cada VMA se corresponde con un conjunto de páginas que es donde se carga el proceso.
- Los segmentos determinan las propiedades que tendrán las páginas en la memoria (read-only, write, execute)
- Corresponde con la Program Header.
- Tabla de símbolos:
 - Es una lista de todos los símbolos (funciones, variables, etc.) que se definen o referencian dentro del archivo.
 - Definida en el header y sus valores están en una section con un tipo específico.
 - Cada entrada en la tabla es una estructura:
 - Nombre, dirección, tamaño, visibilidad (local, global), etc.
 - Se definen en 2 secciones:
 - “.dynsym”: Tabla de Símbolos para el linking dinámico.
 - “.symtab”: Tabla de Símbolos del archivo.
 - En ambas tablas hay una dirección que asocia un símbolo y una indicación sobre el tipo de símbolo.
 - El tipo de símbolo permite acceder a la tabla de strings
- Tabla de Strings:
 - Lista de los strings que se utilizan dentro del archivo.
 - Se definen en 2 secciones:
 - “.dynstr”: Tabla de Símbolos para el linking dinámico.
 - “.strtab”: Tabla de Símbolos del archivo.
 - Se almacenan los nombres de las sections, de los símbolos, etc.
 - Las tabla de símbolos en el campo del nombre contiene un índice a esta tabla de strings.

Relocations

- Proceso de conectar referencias simbólicas con sus definiciones reales.
- Cuando un programa llama a una función, la llamada a instrucción tiene que transferir el control a la correspondiente dirección de memoria donde termina residiendo en tiempo de ejecución esa aplicación.
- Los llamados a funciones externas necesitan ser resueltos durante la ejecución por lo que se necesita información de su ubicación.
- Esta información es necesaria durante el proceso linking dinámico.
- Se almacena en las sections “.rel#name” y “.rela#name”.
- La llamada se termina compilando a una serie de instrucciones:
 - Se tiene una dirección que está en una sección “.plt” que tiene el código que permite saltar al registro de relocación que es donde se está copiando el código de la implementación de la llamada.
 - En “.plt” hay instrucciones que indican un llamado a Dinamic Linker, que cuando se ejecuta actualiza la entrada de la “.got” colocando la dirección donde se cargó la rutina a la que se llama así la próxima vez que se invoque la dirección ya está resuelta.
 - Entonces “.got” tiene la dirección donde termina estando cargada la rutina y se evita realizar todo lo que se dice en el primer punto.
 - Todas las llamadas a rutinas externas se resuelven bajo demanda en tiempo de ejecución cuando se utiliza la función y su código es subido al espacio de memoria
 - “.got” se almacena en un segmento de escritura, la sección “.plt” está solo Read Only.

Portable Ejecutable (PE)

- Formato adoptado para la representación de archivos ejecutables.
- Desarrollado por Microsoft.
- Soporta Dynamic linking y Dynamic loading.
- Facilita la creación de librerías compartidas.
- La única diferencia entre librerías y archivos ejecutables es un bit que indica cómo debe tratarse el archivo. La estructura es igual.

- La versión para 64 bits se la denomina PE32+, la estructura es exactamente la misma solo que agranda o achica algunos campos.
- Representación:
 - La información está representada por un conjunto de estructuras de datos que son las mismas que se utilizan para llevar el archivo a la memoria.
 - Su estructura se encuentran definidos en WINNT.H (Archivo de cabeceras/headers)
- Compuesto por un conjunto de headers y secciones que indican al Loader como cargar el archivo en la Memoria.
- Una vez que está cargado en memoria al archivo se lo conoce como modulo.
 - La dirección de comienzo donde el archivo es cargado se llama HMODULE y a partir de esta se puede acceder a las estructuras del archivos.
- Cada sección del archivo representa información diferente y tiene un conjunto distinto de niveles de protección.
 - Cada sección se almacena en una página aparte.

Secciones

- División Lógica dentro del archivo que contiene código o datos.
- Los datos no necesariamente son solo variables del programa, puede ser información que indica como trabajar con el archivo.
- Cada sección tiene:
 - Un nombre que describe su contenido (no es tenido en cuenta por el SO).
 - Un tipo.
 - Atributos (R,W,X).
- Cada sección dos direcciones
 - Una en el archivo (donde se encuentra)
 - Una dirección virtual (donde se cargará en el espacio de usuario).

RVAs

- Relative Virtual Address.
- Cada archivo tiene un dirección “preferida” de carga en el espacio de direcciones (Virtual):
 - No se puede garantizar que será cargado ahí.

- Para solucionarlo se utilizan RVAs con el fin de direccionar elementos del archivo.
- Es un desplazamiento relativo en memoria a la dirección de inicio donde ese archivo es cargado en la memoria virtual.

Estructura

- Formato de los archivos PE:
 - Header MS-DOS:
 - Todo archivo PE comienza con un pequeño programa MS-DOS ejecutable que quedo por compatibilidad.
 - Entre sus campos hay 2 importantes que distinguen que el archivo es un PE: e_magic y e_lfanew este último es un desplazamiento donde comienza la estructura IMAGE_NT_HEADERS.
 - IMAGE_NT_HEADERS:
 - Especifica los detalles del contenido del archivo.
 - Existe una versión de 32 bits y otra de 64.
 - Algunos se sus campos:
 - Signature.
 - FileHeader: contiene información básica del archivo. Tiene un tipo específico que es IMAGE_FILE_HEADER.
 - Campos importantes:
 - El que dice que tipo de maquina (Machine).
 - Indica la cantidad de secciones definidas (NumberOfSections).
 - Cuando fue creado el archivo (TimeStamp).
 - Características (Characteristics)
 - OptionalHeader: divide su información en 3 grandes grupos
 - Esta está presente en la mayoría de los archivos
 - Puede no estar en las DLLs, si en los ejecutables
 - Alguno de sus campos:
 - Primer grupo de información:

- Suma del tamaño de todas las secciones de código (SizeOfCode)
 - Suma del tamaño de todas las secciones de datos inicializados (SizeOfInitializedData)
 - Suma del tamaño de todas las secciones de datos no inicializados (SizeOfUninitializedData)
 - Segundo grupo:
 - MinorOperatingSystemVersion y MajorOperatingSystemVersion
 - Tercer grupo:
 - IMAGE_DATA_DIRECTORY tiene ubicaciones a direcciones que se consideran importantes dentro del archivo.
- Section Table
 - Es un arreglo de estructuras del tipo IMAGE_SECTION_HEADER.
 - Describe todas las secciones del archivo.
 - La cantidad de entradas se encuentra especificada en IMAGE_NT_HEADER.
 - Campos de IMAGE_SECTION_HEADER:
 - Nombre de la sección (Name[8]).
 - Indica el tamaño de sección (VirtualSize).
 - Indica la RVA donde la sección comienza en la memoria (VirtualAddress).
 - Características (Characteristics).
- Sections
 - Secciones importantes además de .data (donde se almacenan los datos) y .tex (donde está el código):
 - Exports:
 - Información de códigos o datos que pueden ser exportados desde el archivo.

- Nombres de funciones, variables que pueden ser referenciados desde otro archivo.
- Elementos que se exportan se denominan “Símbolos” y se los suele acceder por nombre.
- Se tiene:
 - Un nombre del símbolo.
 - Un número único que lo identifica.
 - Dirección del símbolo dentro del archivo.
- Imports:
 - Símbolos importados desde otro archivo.
- Resources:
 - Almacena información de recursos.
 - Generalmente se almacenan en una sección llamada “.rsrc”.
 - Las estructuras usadas para representar los recursos permite armar una jerarquía similar a la de un filesystem.
- Base Relocatos:
 - Contiene información para el loader.
 - Indica que direcciones deben ser modificadas cuando el archivo es cargado en una dirección que no es su “preferida” (RVAs).

Protección y Seguridad

- Los recursos informáticos deben protegerse.
- Los recursos a proteger son (entre otros):
 - Datos/Información
 - CPU, Memoria, dispositivos
- Uno de los responsables es el SO.
- En base a la aplicación de los mecanismos de protección, se puede determinar el nivel de Seguridad del sistema.

Protección

- Mecanismos del SO para resguardar la información dentro de una computadora, para controlar el acceso de los procesos (o usuarios) a los recursos existentes.
- Mecanismos en dos capas:
 - Acceso al sistema
 - Se resuelve a través de:
 - Autenticación.
 - Control sobre una base de datos de usuarios.
 - Acceso a recursos en un sistema
 - Se resuelve a través de:
 - Permisos.
 - Control de acceso.

Seguridad

- Medida de la confianza en que se puede preservar la integridad de un sistema y sus datos.
- Utiliza distintos mecanismos con el fin de proteger y garantizar ante:
 - Amenazas
 - Confidencialidad de los datos (Intercepción / Modificación)
 - Integridad de los Datos (Modificación)
 - Disponibilidad (Interrupción)
 - Intrusos.
 - Acceso indebido al sistema o datos
 - Perdida Accidental de Datos.
 - Accidentes Naturales
 - Errores de HW o SW
 - Errores humanos

Políticas ≠ Mecanismos:

- Definir políticas antes que los mecanismos.
- Las políticas (qué) definen lo que se quiere hacer, en base a los objetivos.

- Los mecanismos (cómo) definen cómo se hace. Configuraciones e implementaciones reales.
- Diferentes mecanismos para cumplir una política.

Objetos y Dominios

- Un sistema informático es una colección de procesos (los que trabajan con lo que se quiere proteger) y objetos (lo que quiero proteger).
- Objetos de software o de hardware que deben tener un identificador único que permita referenciarlos.
- Procesos pueden realizar un conjunto finito de operaciones sobre objetos.
- Un dominio es un conjunto de pares (objeto, derecho) que especifican el nivel de privilegio que se tiene sobre el objeto determinado. Cada par especifica un objeto y un subconjunto de operaciones que se pueden realizar con él.
- Un derecho significa autorización para efectuar una operación sobre un objeto determinado.
- Quienes pueden ejecutarse dentro de un dominio:
 - Un usuario y define qué puede hacer ese usuario.
 - Un proceso y define el conjunto de objetos a los que podrá acceder.
 - Un procedimiento y definirá el conjunto de variables a las que puede acceder.

Dominios y procesos

- Principio de need-to-know o POLA (Principle of least authority):
 - Todo lo que no está permitido está denegado por defecto.
 - Se enumeran solo los privilegios.
 - No conceder una mayor cantidad de privilegio que los que el proceso realmente necesita para llevar adelante su ejecución.
- La relación entre un proceso y un dominio puede:
 - Estática: un proceso se ejecuta siempre en el mismo dominio ya que el conjunto de objetos a los que accede es fijo.
 - Puede generar que los procesos tengan más privilegios que los que necesitan.

- Dinámica: un proceso puede cambiar de dominio en base a las necesidades del momento.
- Se pueden solapar privilegios entre dominios.

Matriz de acceso

- Controla la pertenencia de objetos a dominios y sus derechos.
 - Las filas representan dominios
 - Las columnas representan objetos.
 - Cada elemento $\text{access}(i,j)$ representa el conjunto de operaciones (derechos) que un proceso puede invocar en un objeto O_j dentro del dominio D_i .
 - Implementa las políticas de protección de un sistema

Objeto Dominio	File1	File2	File3	Printer
D1	Read	Read		Print
D2		Read	execute	print
D3	Read/write	Read/write		

- Operación en la matriz: switch
 - Un proceso puede cambiar de un dominio a otro.
 - Para poder cambiar de un dominio a otro se debe habilitar la operación switch sobre un objeto (dominio). Tanto el dominio como la matriz son objetos.
 - La matriz en sí es un objeto que posee atributos. De esta manera se define si sobre un dominio se realiza asignación estática o no:
 - La conmutación del dominio D_i al dominio D_j estará permitida si se encuentra definida en el switch $\text{access}(i,j)$

Objeto Dominio	File1	File2	File3	Printer	D1	D2	D3
D1	Read	Read		Print		switch	
D2		Read	execute	print			
D3	Read/write	Read/write			switch		switch

- Operación copy
 - Es un derecho que se asocia a un elemento $\text{access}(i,j)$ de la matriz.
 - Indica que un proceso ejecutándose en ese

Objeto Dominio	File1	File2	File3	Printer
D1	Read	Read		Print
D2		Read *	execute	print
D3	Read/write			
Objeto Dominio	File1	File2	File3	Printer
D1	Read	Read		Print
D2		Read *	execute	print
D3	Read/write	Read		

dominio puede copiar los derechos de acceso de ese objeto dentro de su columna

- Se denota con *.
- Se replica el permiso a cualquier otro dominio.
- Privilegio que se da para el usuario que es owner del archivo.
- Variantes:
 - Transferencia: si un derecho se copia desde $\text{matriz}(i,j)$ a $\text{matriz}(k,j)$, el derecho desaparece para $\text{matriz}(i,j)$, o sea, el derecho fue transferido.
 - Propagación o copia limitada: Se copia el derecho pero no el derecho a copia en el nuevo (R^* es copiado como R , no como R^*).

- Operación owner

- Permite agregar nuevos derechos y borrar ya existentes.
- Si $\text{matriz}(i,j)$ incluye el derecho de owner entonces un proceso ejecutándose en el dominio D_i puede agregar y borrar cualquier entrada en la columna j .

Objeto \ Dominio	File1	File2	File3	Printer
D1	Read	Read		Print
D2		Read * owner	execute	print
D3	Read/write			
Objeto \ Dominio	File1	File2	File3	Printer
D1	Read	Read/write		Print
D2		Read * owner	execute	print
D3	Read/write	Read		

- Operación Control

- Indica que pueden modificarse y borrarse derechos dentro de una fila.
- La operación control es aplicable sólo a objetos dentro del mismo dominio.
- Si $\text{matriz}(i,j)$ incluye el derecho de control, entonces un proceso ejecutándose en el dominio D_i puede remover cualquier derecho de acceso dentro de la fila j .
- Cualquier proceso que se ejecute en el dominio D_1 , puede controlar los Switch Access de cualquier columna del dominio D_3 .

Objeto \ Dominio	File1	File2	File3	Printer	D1	D2	D3
D1	Read	Read		Print		switch	control
D2		Read	execute	print			
D3	Read/Write	Read/write			switch		switch

- Resumiendo:

- Copy, owner y control son operaciones que se utilizan para controlar cambios al contenido de la matriz de acceso.

- Switch y Control: son aplicables sólo a dominios.
- Copy y owner: puede modificar derechos dentro de una columna.
- Control: puede modificar derechos dentro de una fila
- Implementación:
 - No es bajo el formato de matriz porque es ineficiente. Se pueden tener muchos elementos vacíos.
 - Se almacenan solo los elementos ocupados utilizando 2 métodos:
 - Almacenarla por filas.
 - Almacenarla por columnas.
 - Se debe optimizar el acceso.
 - Alternativas de implementación:
 - Tabla Global:
 - La más sencilla.
 - Consiste en conjunto de tuplas.
 - Cada vez que se ejecuta una operación M sobre un objeto O_j sobre el dominio D_i , se analiza la tabla y se verifica si se encuentra una terna $\langle D_i, O_j, M \rangle$.
 - Si se encuentra se permite la operación, si no se deniega.
 - El tamaño de la tabla hace que no se pueda almacenar toda en memoria.
 - Lista de Control de Acceso por objetos:
 - Se asocia cada objeto con una lista (ordenada) que contiene todos los dominios que pueden acceder al objeto, y la forma de hacerlo.
 - Cada columna de la matriz se puede ver como una lista de acceso a un objeto.
 - Para cada objeto, hay una lista de pares ordenados $\langle \text{dominio}, \text{derechos} \rangle$
 - Cuando se intenta realizar una operación M sobre un objeto O_j en el dominio D , se busca en la lista en el objeto O_j una entrada, donde M pertenece al conjunto R_k .
 - Permisos están el espacio del Kernel, la pertenencia al dominio en el espacio del Usuario.

- Lista de Capacidades por Dominio:
 - Es una lista de objetos del dominio con sus derechos (división por filas)
 - Se le conoce como lista de capacidades y a los elementos individuales que tiene como capacidades
 - El proceso no accede directamente a la lista.
 - Esta lista es un objeto protegido, a la que sólo accede el SO
 - Cada proceso tiene una lista con los objetos que puede utilizar, junto con que operaciones (el dominio)
 - Compleja de mantener cuando se quiere revocar o modificar un permiso sobre un objeto
 - Se tiene que recorrer todas las listas de capacidades

Sistemas confiables

- Otro factor que afecta el nivel de seguridad de un sistema es el código:
 - Código mal intencionado: Virus, gusanos, troyanos, etc.
 - Código con errores de programación: Backdoors
- El gran enemigo de la seguridad es el agregado de funcionalidad:
 - Un sistema minimalista muy seguro pobremente usable.
 - Un sistema con muchas características más errores y más posibilidad de ser vulnerado.
 - No hay seguridad al 100%.
- Explotación de errores en código:
 - Los procesos, junto con el Kernel son una potencial amenaza a la seguridad de un sistema.
 - Los atacantes aprovechan errores en la codificación del SO, o algún proceso con alto nivel de privilegios con el fin de que los mismos cambien su funcionamiento normal:
 - Desbordamiento de buffer.
 - Tiene como objetivo sobrescribir datos de ciertas zonas de memoria intencionalmente

- Se trata de un error del software. Se quiere copiar una cantidad de datos más grande que el área definida.
- Si el error no pudiera ser detectado en runtime, podría alterarse el flujo del programa.
- Si la posición de memoria se sobrescribe con un valor sensible, la ejecución podría encadenar una nueva ejecución de un programa malicioso
 - Shellcode: código copiado especialmente preparado para obtener los privilegios del programa atacado.
 - Ejemplo: Ejecución en una pila.
 - Si los datos proporcionados superan cierta cantidad de datos (capacidad del buffer), el buffer podría seguir creciendo y se podrían sobrescribir áreas de memoria con “posibles datos sensibles”:
 - Si se sobrescribe la dirección de retorno, se podría saltar a una dirección aleatoria.
 - Si se sobrescribe inteligentemente, se podría lograr la ejecución de código malicioso.
 - Si el programa que se ataca es SETUID a root en Unix, el Shell code ejecutado se ejecutaría como root.
- Cadenas de formato.
 - Puede ocurrir que lo que se ingresa por teclado una cadena que incluya caracteres que hagan que el printf funcione de modo no deseado haciendo que, por ejemplo, se sobrescriba la dirección de retorno de printf y se salte a donde se quiera. Si el programa se ejecuta con SETUID root, se podría saltar a un Shell code que se ejecute con esos privilegios.
- Retorno a libc.

- Desbordamiento de enteros.
- Inyección de código.
 - No se están validando correctamente las cadenas que se están ingresando como parámetro lo que podría llegar a permitir una inyección de código muy común en la web.